Fala Cidade

Atualização para o Expo SDK 58 e configuração do mapa

Passo 0 - Criar a branch da aula

```
git checkout -b aula26202026
```

Cada assunto novo ganha a sua branch. Assim a *main* continua funcionando.

Parte 1 - Atualização do Expo SDK 57 para o 58

O projeto estava no SDK 57. O SDK 58 já é a versão *latest* do Expo.

Pacote	Antes (SDK 57)	Depois (SDK 58)
expo	57.0	58.0.19
react-native	0.86.3	0.88.0
react / react-dom	19.2.3	19.3.0
expo-router	~57.0.24	~58.0.26
react-native-maps	1.27.2	1.29.11
react-native-reanimated	4.5.1	4.7.1
react-native-gesture-handler	~2.32.0	~3.3.0
react-native-worklets	0.10.1	0.13.0

Passo 1 - Atualizar as dependências

```
npx expo install expo@^58.0.0 --fix
```

Esse comando falhou com um conflito de dependência: o npm ainda tinha o pacote transitivo **@expo/ui 57**, e o **expo-router 58** pede a versão 58 (erro ERESOLVE).

Passo 2 - Reinstalar do zero

```
rm -rf node_modules package-lock.json
npm install
npx expo install --fix
```

O *package-lock.json* é gerado de novo, então o diff dele fica grande. O npm também avisa que bloqueou scripts de instalação de 3 pacotes (unrs-resolver, @firebase/util, protobufjs).

Passo 3 - Verificar

```
npx expo-doctor        # 20/20 checks passed
npx tsc --noEmit
npm run lint
npx expo export --platform ios
npx expo export --platform android
npx expo export --platform web
```

Passo 4 - Corrigir a rota inicial (app/_layout.tsx)

O SDK 58 passou a validar a rota inicial. O layout apontava para **login**, que não existe (a tela de login é *app/index.tsx*). O erro aparecia no export web: *The initial route name "login" was not found*.

```
// antes
export const unstable_settings = {
  //anchor: "(tabs)",
  initialRouteName: "login",
};

// depois
export const unstable_settings = {
  anchor: "index", // tela de login (app/index.tsx)
};
```

Passo 5 - Refazer o development build

O build instalado no aparelho continua no SDK 57. Gere um novo e limpe o cache do Metro:

```
npx expo start -c
npx expo run:ios      # ou: npx expo run:android
```

Parte 2 - Configuração do mapa atual

A tela **Mapa** pede a localização do usuário, descobre cidade e estado, centraliza o mapa desenha pinos de ocorrências coloridos por status, com busca, filtros e botão de novo reporte.

Passo 1 - Bibliotecas

```
npx expo install react-native-maps expo-location
```

Use *expo install* (e não *npm install*) para pegar a versão compatível com o SDK.

Passo 2 - Permissão de localização (app.json)

```
[
  "expo-location",
  {
    "locationWhenInUsePermission":
      "O app usa sua localização para mostrar os buracos da sua cidade."
  }
]
```

Passo 3 - Contexto da cidade

context/cidade-context.tsx pede a permissão, lê o GPS e converte coordenadas em cidade/UF (*reverseGeocodeAsync*). Expõe *cidade, uf, coordenadas, status, atualizar*. O *status* pode ser: carregando, pronto, negado ou erro. Use o hook **useCidade()**. O *CidadeProvider* envolve o app dentro do *AuthProvider* em *app/_layout.tsx*. A sigla do estado vem de **constants/estados.ts**.

Passo 4 - Componente do mapa

components/mapa-view.tsx usa *MapView* com *showsUserLocation* e *initialRegion* (zoom 0.02 ≈ um bairro). Na web, **mapa-view.web.tsx** mostra só um aviso, pois o react-native-maps não roda no navegador. A tela usa *key* no mapa para recriá-lo quando o GPS chega.

Passo 5 - Pinos de ocorrências por status

Arquivo	Função
types/ocorrencia.ts	Tipos Ocorrencia e StatusOcorrencia (aberto, em_execucao, resolvido).
constants/status.ts	Rótulo e cor de cada status (vermelho, âmbar, verde).
services/ocorrencias.ts	buscarOcorrencias(centro): 6 ocorrências de mentira em volta do usuário. Trocar por fetch em GET /occurrences quando a API existir.
hooks/use-ocorrencias.ts	Carrega as ocorrências das coordenadas atuais.
components/mapa-view.tsx	Prop ocorrencias: um Marker por item, com pinColor do status.
app/(tabs)/mapa.tsx	Usa o hook, passa as ocorrências ao mapa e mostra a legenda.

Passo 6 - Campo de pesquisa e filtros

components/busca-filtros.tsx (novo) desenha o campo *Buscar ocorrência* e os botões **Todos, Abertos, Em execução e Resolvidos**. O botão ativo fica verde e o "x" limpa a busca. O tipo *Filtro* é "todos" ou um status.

Em **app/(tabs)/mapa.tsx** a tela guarda dois estados (*busca* e *filtro*) e calcula a lista que aparece no mapa:

```
const normalizar = (t: string) =>
```

```
  t.normalize("NFD").replace(/[\u0300-\u036f]/g, "").toLowerCase();
const termo = normalizar(busca.trim());
const visiveis = ocorrencias.filter(
  (o) =>
    (filtro === "todos" || o.status === filtro) &&
    normalizar(`${o.titulo} ${o.descricao ?? ""}`).includes(termo),
);
```

A busca ignora acento e maiúscula e olha título e descrição. O mapa recebe *visiveis* no lugar de *ocorrencias*. Se nada combinar, aparece "Nenhuma ocorrência encontrada.". A busca só esconde pinos: ela não centraliza o mapa no resultado.

Passo 7 - Botão de novo reporte e tela Novo reporte

Arquivo	Função
app/(tabs)/mapa.tsx	Botão redondo verde com "+" no canto inferior direito do mapa. Chama router.push("/novo-reporte"). Sobe quando o aviso de localização aparece.
app/novo-reporte.tsx	Tela nova: só o texto "Novo reporte" no centro. Nada mais por enquanto.
app/_layout.tsx	Registra a tela no Stack, mostrando só a seta de voltar (sem título).

Os arquivos antigos *criacao_reporte.tsx* e *criarReport.tsx* são rascunhos que nenhuma tela usa.

Passo 8 - Testar

- O **react-native-maps não roda no Expo Go**: use um development build (*npx expo run:ios* ou *run:android*).
- Ao abrir, o celular pergunta se pode usar a localização.
- Permitindo: o título mostra a cidade, o mapa centraliza e aparecem os pinos e a legenda.
- Negando: aparece o aviso com o botão "Abrir ajustes".
- Na web (*npx expo start --web*): só o aviso de que o mapa é do app.
- **Android:** o mapa do Google fica cinza sem uma chave da Google Maps API (*android.config.googleMaps.apiKey* no app.json). Essa chave ainda não está configurada.

Próximos passos

- Montar o formulário da tela Novo reporte (foto, descrição, local).
- Trocar o mock pela API real (URL, formato do JSON e autenticação ainda não definidos).
- Seletor de cidade.
- Centralizar o mapa no resultado da busca.